

Hashing: The Foundation of Data Integrity

 zihad.com.bd/posts/certs-ins-and-outs

May 1, 2025



Learn how hashing transforms data into fixed-length, secure outputs and why it's crucial for authentication, integrity checks, and password storage.

Understanding hashing and digital certificates is essential for any modern developer. This guide breaks down these core security concepts in a clear, practical way.

Some basics

Hashing “#”

Hashing is the process of transforming a character string to an another format using a mathematical function which makes it:

- fast -> using trapdoor functions which are easy to calculate but hard to reverse e.g.: product of two prime number but finding out which two numbers were they is much much harder
- theoretically unreversible
- provides the same output for the same input meaning A always will produce B
- prevents collisions -> cannot produce the same output for the different input meaning A -> B but C never will produce B
- reduces the input size -> it divides the input to equal blocks then compress them before running the hash function to run the hashing function on it -> the output will be always the same size

Ok, but what is it good for?

Usually hashes are sent along side with a payload/data to determine if it was tempered after the sender initially sent it to the client. But they are also used for authentication when the user's password won't be stored in the database just the **salted** hash itself.

Salting 🧂 is the process of concatenating an already known property of the user to the password and hash it together. This is important to prevent [rainbow table attacks](#) for leaked databases which contains the hashes of the user passwords.

For example each certificate contains the hash fingerprint of itself and this way when client receives it, then it can be hashed again and see if the fingerprints are matching.

Pepper 🌶️ is one more additional layer of security when storing hash. It is basically doing one more round of hashing but now using some shared secret which will be the same for all of the entries in the database.

Hashing algorithms

There are multiple algorithms which are used for hashing, but they have different characteristics. Some of them lost their reliability by proving it can create collisions when we are doing lot of hashes, or simply the computers evolved so much that the hash can be reversed because it is not using big enough prime numbers or complexity.

Lets see which algo should be used in 2025

Algorithm	Suggested in 2025	Digest Size	Speed	Security Level	Typical Applications
MD5	❌ No	128 bits	Very Fast	Weak (prone to collisions)	Checksums, basic data integrity
SHA-1	❌ No	160 bits	Fast	Weak (collisions found)	Legacy systems, data integrity
SHA-2 (SHA-256)	✅ Yes	256 bits	Moderate	Strong	Cryptographic protocols, blockchain, SSL
SHA-2 (SHA-512)	✅ Yes	512 bits	Slower	Very Strong	High-security applications, SSL

Algorithm	Suggested in 2025	Digest Size	Speed	Security Level	Typical Applications
SHA-3 (SHA3-256)	✔ Yes	256 bits	Moderate	Very Strong	Cryptographic protocols, secure storage
SHA-3 (SHA3-512)	✔ Yes	512 bits	Slower	Very Strong	High-security applications
BLAKE2	✔ Yes	Variable (up to 512 bits)	Very Fast	Strong	General purpose, cryptography, file hashing
BLAKE3	✔ Yes	256 bits	Very Fast	Strong	File hashing, cryptography, real-time systems
RIPEMD-160	⚠ Limited use	160 bits	Moderate	Moderate	Digital signatures, blockchain
Whirlpool	✔ Yes	512 bits	Slow	Very Strong	High-security storage, cryptographic protocols
Tiger	⚠ Limited use	192 bits	Fast	Moderate	Data integrity verification
Argon2	✔ Yes	Variable	Slow (memory-intensive)	Very Strong	Password hashing
PBKDF2	✔ Yes	Variable	Slow (iteration-based)	Very Strong	Password hashing, key derivation
Scrypt	✔ Yes	Variable	Slow (memory-intensive)	Very Strong	Password hashing, key derivation

Never store password in the database plaintext, use a hashed and salt&peppered string preferably using at least sha2 with multiple iterations/work factors. This way the cracking process itself won't be worth for the attacker.

What is a certificate?

A **certificate** or **digital certificate** is a unique, [digitally signed](#) document which authoritatively identifies the identity of an individual or organization. Using [public key cryptography](#), its authenticity can be verified to ensure that the software or website you are using is legitimate. On the Internet, a certificate is signed by a trusted [CA](#) (certificate authority), and verified with the authority's public key. The decrypted certificate contains a verified public key of the certificate holder (website operator), with which encrypted [HTTPS](#) communications can be established. [source](#)

So basically the certificate is a unique generated chain of characters in a specific format which can be validated by a third party (CA) therefore can be used for authentication/authorization and prove the integrity of the document.

Use cases

Certificates are used for:

- securing communication between two parties -> like ssh or https
- proving client access rights for a server -> mutual tls (mTLS)
- proving integrity of a document -> digitally signed pdf

How to use certificate to secure communication?

The smart people invented a system which relies on certificates to secure the data using **public-key cryptography** or in another name -> **Asymmetric cryptography**

Algorithms for certificate generation and key sizes

The certificates usually generated with one of the following algorithms in combination with one of the previously mentioned hashing algorithms.

Key is basically a random string of bits that serves as input for cryptographic algorithms. The stronger is the key and the encryption algorithm your data is in bigger safety.

- RSA
 - it is using big prime numbers and a factoring trap door function, bigger keys provide greater security
 - common key size are 2048 and 4096. 2028 bit according to NIST is secure until 2030
- ECDSA (Elliptic curve)
 - based on mathematical properties of [elliptic curves](#) from which next point can be calculated easily (public key), but reversing it almost impossible -> using shorter keys due to it
 - 256 bit key size is equivalent of 3072 bit RSA key size
- DSA
 - it is using discrete logarithmic properties and private key generated from the content that we want to sign -> used for ensuring file integrity and verification

How to generate our key pair using RSA?

First generate the private key

```
openssl genpkey -algorithm RSA -out private_key.pem -pkeyopt
rsa_keygen_bits:2048
```

Generate the public key

```
openssl rsa -in private_key.pem -pubout -out
public_key.pem
```

Raw content:

| File: private_key.pem

```
1 | -----BEGIN PRIVATE KEY-----
2 | MIIEvQIBADANBgkqhkiG9w0BAQEFAASCBCwggSjAgEAAoIBAQDNSqlwtI4exogf
3 | REW0vSkYUVAIva6qblvCJlhe0Eyllek+RKGPQI4Iw/bgAg8UFb2Gqm06Aci8aT7r
4 | e0+acZI10FKX1pXpSJ0mQ7tLuDhCoM/LYr+lvxqHaq/T/u+8jLNds2Iw20rhfbdD
5 | huJ/cYCw5N/8qdxsmph7tuvwpyDl6pAV7YpxE4RZVld7SAxQtAjoW1TSgGRFDkiu
6 | DKMwGur8Y4zz+ZeJuf93gBt0/St/IGWwLh1vXGoSI60yik0ZCKc8lN++kYLJ846y
7 | p7G2Gqang6EKs3N90IbfcFhkwL65ZBddhUVbQD6aSl/Pfn3T2plek8H3BxYAf0us
8 | qpfzfA29AgMBAECggEAAwfYsqK/vF2S8d84pah0PIEqzM48D2EMsruQ9uce3ozt
9 | ghFST0KVNjTqZpkxLss5Xo0SZtMS6xEksCZEYRPPJED+Gya8VNLRGxkB4VbJyDZs
10 | 6uQ8z0qCqjJVgSwzwthG55m319/M5bJM9UF/A4Gh9DqUwe6iBepsWNS09XsKJcq/
11 | a4AdnVGRBn+tldCVCK+rGgIKV0mAZyIDERK5LAXDG6Uo09i1UodA8ofGg2WHRBqN
12 | eDoc5/XWEjdAUaHPhgpGIpjtfHuXcKzLLlsLBVyJ5t8A/MQgAzWYbkuiL+R6QyFZ
13 | Uj751bMNNstNhSQrN8PwxHOBBS43EmqRp+DiUDlPoQKBgQDq1D0gX9qqyqBAqvqJ0
14 | C/A3vY4ESLybUZBbqQt48H3TivRc+ZmY5Cdn+UjNAKL7LiWCz2H9wEhZP7XWZmz
15 | ZYS6NX8j1Q0fseuqNJyV7E0W3ddtARDRp6gTSCk8SSEn+gBPh748iVvHYI4+BZm
16 | pMf5F3CrLaWjDUI/q0r2Ifk8kQKBgQDfzMOKsknL3DgYfyY67SEDH1ILMUxEnabi
17 | zliuvbZzrjyYIUlTpyPdAe14vrGYIjT2oYzweRYVMq5q+ug0Dsohc9GT2WsnXTBI
18 | LHM2JC9eQ/FNGmHYt+g3KlsSK93YFiXW/Zwj8lRiP6ent1VvqQLfrZj5w6EmLBGB
19 | 8q2BgjsEbQKBgFbm8rgN4QQul9UJuVrStDIcjrvc5Bdihta05075SMdJbAcgQ4xG
20 | GgMqQJtc+f06UxWb0UiIyz1Nf9W4Dd5K2mSCuatMAExP7p0NZslApv6k2D05oySP
21 | FnzCLit8JfSCLwQ/itIiTvmorNsc/XGU07GMubWPK7rkDG5tUMC7ZZRAoGBANpc
22 | J86D9oz56aG9LcgA5lztildxWkD4sYRLWvPKFsJ4DG6Uy77bLZrN7q4JIA6WcabS
23 | KXnYKzuzn4qyFQjiVUYegzYGvrcTJvDGQ+hnlfAbyqy40RADIIIfzvYAVXPchZPpH
24 | omG8Z6kVArgZoTq5ZRxpQD1Z58EibCI39GLB4BhAoGAML+iU0yUj5FZs/X8vLPx
25 | ZJwuZU+7GHM1HbJXhH/BZY0poBzBNFjKj8mu5l36T4x+EZ6ySkmfJ5isEdXcK06M
26 | JFukU8YHUMWCN/YZpNg/92XNoB5PtUFj/bsEBHAu1GZzXgcjoA84rcgvLQq+4Ufb
27 | 8xQKC3zQmqhkeq3GewThg48=
28 | -----END PRIVATE KEY-----
```

| File: public_key.pem

```
1 | -----BEGIN PUBLIC KEY-----
2 | MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAzUqtCL50HsaIH0RFjr0p
3 | GFFQCL2uqm5bwidYXjhMpZXPkShj0C0CMP24AIPFBW9hqpjugHIvGk+63tPmnGS
4 | NThSl9aV6UidJk07S7g4QqDPy2K/tb8ah2qv0/7vvIyzXbNiMntK4X25w4bif3GA
5 | sOTf/KncbJqYe7br8Kcg5eqQFe2KcR0EWVZXe0gMULQI6FtU0oBkRQ5IrgyJMBRq
6 | /GOM8/mXibn/d4AbdP0rfyBllyYdb1xqEi0tMopDmQinPJTfvpGJSf00sqexthqM
7 | p40hCrNzfdCG3whYZMJeuWQXXYVFW0A+mkpzf3zd09qZXpPB9wcWAHzrrKqX83wN
8 | vQIDAQAB
9 | -----END PUBLIC KEY-----
```

Dissect it with openssl:

```
openssl pkey -in private_key.pem -text -  
noout
```

Private-Key: (2048 bit, 2 primes)
modulus:

00:cd:4a:ad:70:b4:8e:1e:c6:88:1f:44:45:8e:bd:
29:18:51:50:08:bd:ae:aa:6e:5b:c2:27:58:5e:38:
4c:a5:95:e9:3e:44:a1:8f:40:8e:08:c3:f6:e0:02:
0f:14:15:bd:86:aa:63:ba:01:c8:bc:69:3e:eb:7b:
4f:9a:71:92:35:38:52:97:d6:95:e9:48:9d:26:43:
bb:4b:b8:38:42:a0:cf:cb:62:bf:b5:bf:1a:87:6a:
af:d3:fe:ef:bc:8c:b3:5d:b3:62:30:db:4a:e1:7d:
b9:c3:86:e2:7f:71:80:b0:e4:df:fc:a9:dc:6c:9a:
98:7b:b6:eb:f0:a7:20:e5:ea:90:15:ed:8a:71:13:
84:59:56:57:7b:48:0c:50:b4:08:e8:5b:54:d2:80:
64:45:0e:48:ae:0c:a3:30:1a:ea:fc:63:8c:f3:f9:
97:89:b9:ff:77:80:1b:74:fd:2b:7f:20:65:96:96:
1d:6f:5c:6a:12:23:ad:32:8a:43:99:08:a7:3c:94:
df:be:91:89:49:f3:8e:b2:a7:b1:b6:1a:a6:a7:83:
a1:0a:b3:73:7d:d0:86:df:08:58:64:c2:5e:b9:64:
17:5d:85:45:5b:40:3e:9a:4a:5f:cf:7c:dd:d3:da:
99:5e:93:c1:f7:07:16:00:7c:eb:ac:aa:97:f3:7c:
0d:bd

publicExponent: 65537 (0x10001)
privateExponent:

03:07:d8:b2:a2:bf:bc:5d:92:f1:df:38:a5:a8:4e:
3c:81:2a:cc:ce:3c:0f:61:0c:b2:bb:90:f6:e7:1e:
de:8c:ed:82:11:52:4f:42:95:36:34:ea:66:99:31:
2e:cb:39:5e:8d:12:66:d3:12:eb:11:24:b0:26:44:
61:13:cf:24:40:fe:1b:26:bc:54:d2:d1:1b:19:01:
e1:56:c9:61:d6:6c:ea:e4:3c:cf:4a:90:72:32:55:
81:2c:30:ce:d8:46:e7:99:b7:d7:df:cc:e5:b2:4c:
f5:41:7f:03:81:a1:f4:3a:94:c1:ee:a2:05:ea:6c:
58:db:34:f5:7b:0a:25:ca:bf:6b:80:1d:9d:51:ab:
04:df:ad:d5:d0:95:08:af:ab:1a:09:4a:57:49:80:

67:22:03:11:12:b9:2c:0c:43:1b:a5:28:3b:d8:b5:
52:87:40:f2:87:c6:83:65:87:44:1a:8d:78:3a:1c:
e7:f5:d6:12:37:40:51:a1:cf:86:0a:46:22:98:ed:
7c:7b:97:70:ac:e5:2e:5b:0b:05:5c:89:e6:df:00:
fc:c4:20:03:3c:18:6e:4b:a2:2f:e4:7a:43:21:59:
52:3e:f9:d5:b3:0d:36:cb:4d:85:24:2b:37:c3:f0:
c4:73:81:05:2e:37:12:6a:91:a7:e0:e2:50:39:4f:
a1
prime1:
00:ea:d4:33:a0:5f:da:b2:a8:10:2a:be:a2:74:0b:
f0:37:bd:8e:04:48:bc:9b:51:90:5b:a9:0b:78:f0:
7d:d3:8a:f4:5c:f9:99:98:e4:27:67:f9:48:cd:00:
a2:fb:94:88:96:0b:3d:87:f7:01:21:64:fe:d7:59:
99:b3:65:84:ba:35:7f:23:d5:03:9f:b1:eb:aa:34:
9c:95:ec:43:96:dd:d7:6d:01:17:51:a5:fe:a0:4d:
20:a4:f1:24:84:9f:e8:01:3e:1e:f8:f2:25:6f:1d:
82:38:f8:16:66:a4:c7:f9:17:70:ab:2d:a5:a3:0d:
42:3f:ab:4a:f6:21:f9:3c:91
prime2:
00:df:cc:c3:8a:b2:49:cb:dc:38:18:7f:26:3a:ed:
21:03:1f:52:0b:31:4c:44:9d:a6:e2:cf:58:ae:bd:
b6:73:ae:38:d8:21:49:53:a7:23:dd:01:ed:78:be:
b1:98:22:34:f6:a1:8c:f0:79:16:15:32:ae:6a:fa:
e8:0e:0e:ca:21:73:d1:93:d9:64:a7:c5:30:48:2e:
13:36:24:2f:5e:43:f1:4d:1a:61:d8:b7:e8:37:2b:
5b:12:2b:dd:d8:16:25:d6:fd:95:a3:f2:54:62:3f:
a7:a7:b7:55:6f:a9:02:df:ad:98:f9:c3:a1:26:94:
11:81:f2:ad:81:82:3b:04:6d
exponent1:
56:e6:f2:b8:0d:e1:04:2e:97:d5:09:b9:5a:d2:b4:
32:1c:8e:fa:dc:e4:17:62:86:d6:b4:e7:4e:f9:48:
c7:49:6c:07:20:43:8c:46:1a:a3:2a:a8:9b:5c:f9:
f3:ba:53:15:9b:d1:48:88:cb:3d:4d:7f:d5:b8:0d:

de:4a:da:64:82:b9:ab:4c:00:4c:4f:ee:93:8d:66:
c9:40:a6:fe:a4:d8:3d:39:a3:24:8f:16:7c:c2:2e:
2b:7c:25:f4:82:ac:bc:10:fe:2b:48:89:3b:e6:a2:
b3:6c:73:f5:c6:53:4e:c6:32:e6:d6:3c:ae:eb:90:
31:b9:b5:43:02:ed:96:51
exponent2:
00:da:5c:27:ce:83:f6:8c:f9:e9:a1:bd:2d:c8:00:
e6:5c:ed:8a:57:71:5a:40:f8:b1:84:4b:5a:f3:e4:
16:c8:f8:0c:6e:94:cb:be:db:2d:9a:cd:ee:ae:09:
21:ae:96:71:a6:d2:29:79:d8:2b:3b:b3:9f:8a:b2:
15:08:e2:55:46:1e:83:36:06:be:b7:13:26:f0:c6:
43:e8:67:95:f0:1b:ca:ac:b8:39:10:03:20:87:f3:
bd:80:15:5c:f7:21:64:fa:47:a2:61:bc:67:a9:15:
02:b8:19:a1:3a:b9:65:1c:69:7d:00:f5:67:9f:04:
89:b0:88:df:d1:a5:07:80:61
coefficient:
30:bf:a2:53:4c:94:8f:91:59:b3:f5:fc:bc:b3:f1:
64:9c:2e:65:4f:bb:18:73:35:1d:b2:57:84:7f:c1:
65:8d:29:a0:1c:c1:34:58:ca:8f:c9:ae:e6:5d:fa:
4f:8c:7e:11:9e:b2:4a:49:9f:27:98:ac:11:d5:dc:
28:ee:8c:24:5b:a4:53:c6:07:50:c5:82:37:f6:19:
a4:d8:3f:f7:65:cd:a0:1e:4f:b5:41:63:fd:bb:04:
04:70:2e:d4:66:73:5e:07:23:a0:0f:38:ad:c8:2f:
2d:0a:be:e1:47:db:f3:14:0a:0b:7c:d0:9a:a8:64:
7a:ad:c6:7b:04:e1:83:8f

Dissect the public key too:

```
openssl rsa -in public_key.pem -pubin -text -noout
```

Public-Key: (2048 bit)

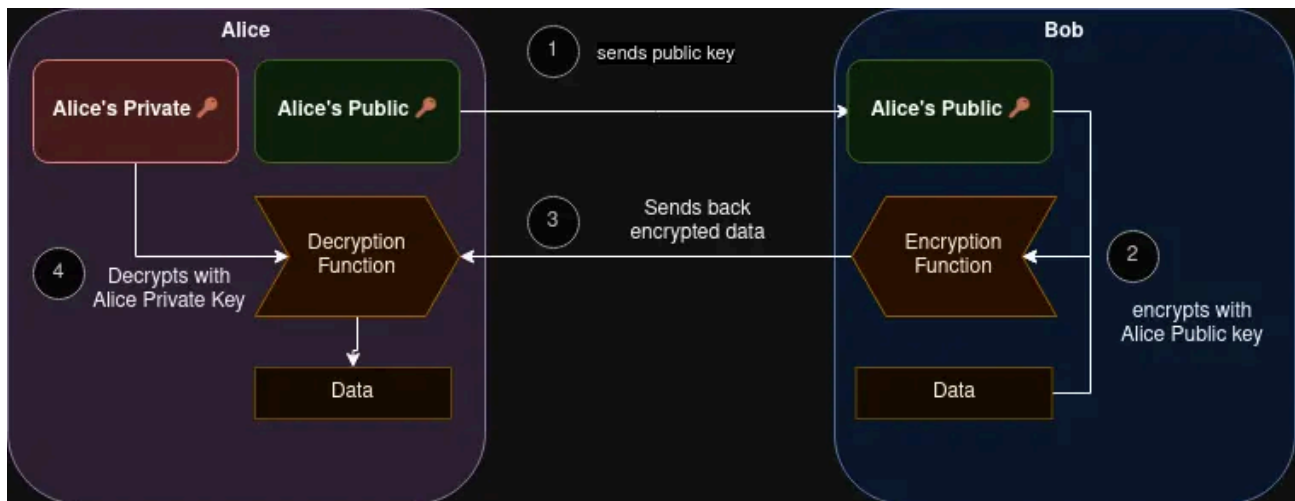
Modulus:

```
00:bb:24:9d:47:ba:81:6d:67:40:16:02:f6:b0:fa:
19:84:bc:35:ed:e8:42:7a:7a:27:de:80:cc:75:2c:
e8:df:a9:05:bf:bf:20:1c:05:e8:52:45:ee:87:61:
a7:c1:93:d9:b9:b0:ad:8d:5b:93:24:4c:3e:65:e7:
89:33:a3:1f:5b:34:02:86:84:00:0c:c5:76:2a:7f:
d4:d4:0b:b2:77:cb:28:79:38:62:59:9e:ed:b2:18:
bb:22:c8:ec:d3:f0:c8:1e:8a:82:b3:9d:cd:69:fa:
62:39:53:46:ec:39:5c:b1:08:4e:ce:ae:86:d3:17:
0b:3d:7f:ff:b9:75:a7:c0:64:f7:12:b3:42:51:17:
03:cc:c9:58:11:23:c6:a1:9e:9a:c6:dd:07:62:29:
51:81:ec:f4:de:66:ed:62:06:78:07:14:d8:5a:ce:
75:ee:eb:17:87:7a:5c:b5:9c:7f:bd:4c:85:90:02:
57:29:4e:c4:28:a7:2f:54:d9:f2:0c:40:94:32:10:
d9:2c:fa:25:92:56:18:42:bd:51:5e:12:dd:e1:d8:
ed:35:8e:58:c4:1e:57:66:0d:2b:e0:de:92:82:f4:
ef:53:ad:3e:2a:93:e4:f3:d9:d7:38:92:60:dd:12:
83:da:c2:18:a6:d5:30:1e:2f:6e:f9:01:0c:11:7a:
28:83
```

Exponent: 65537 (0x10001)

We can see that public and private keys are just the holders of the mathematical product and properties of the key generation process.

The process of securing communication with public and private keys



In a very oversimplified manner this happens:

```
const privateKey = randomString();
const publicKey = generatePublicKey(privateKey);
const plainMessage = 'Hello Mr Bug!🐛';

const encryptedMessage = encrypt(publicKey, plainMessage);
const decryptedMessage = decrypt(encryptedMessage,
privateKey);
console.log(decryptedMessage); //'Hello Mr Bug!🐛'
```

The roles of ● Public and ● Private keys

What one key does

The other will validate

● Public encrypts

● Private decrypts

● Private digitally signs

● Public verifies signature

● Private authenticates

● Public verifies authentication

Key exchange

Since both communicating parties will send their public keys there is no need for any kind of secure key exchange protocol.

Public-Key Infrastructure (PKI)

How the heck validate the cert? 🤔

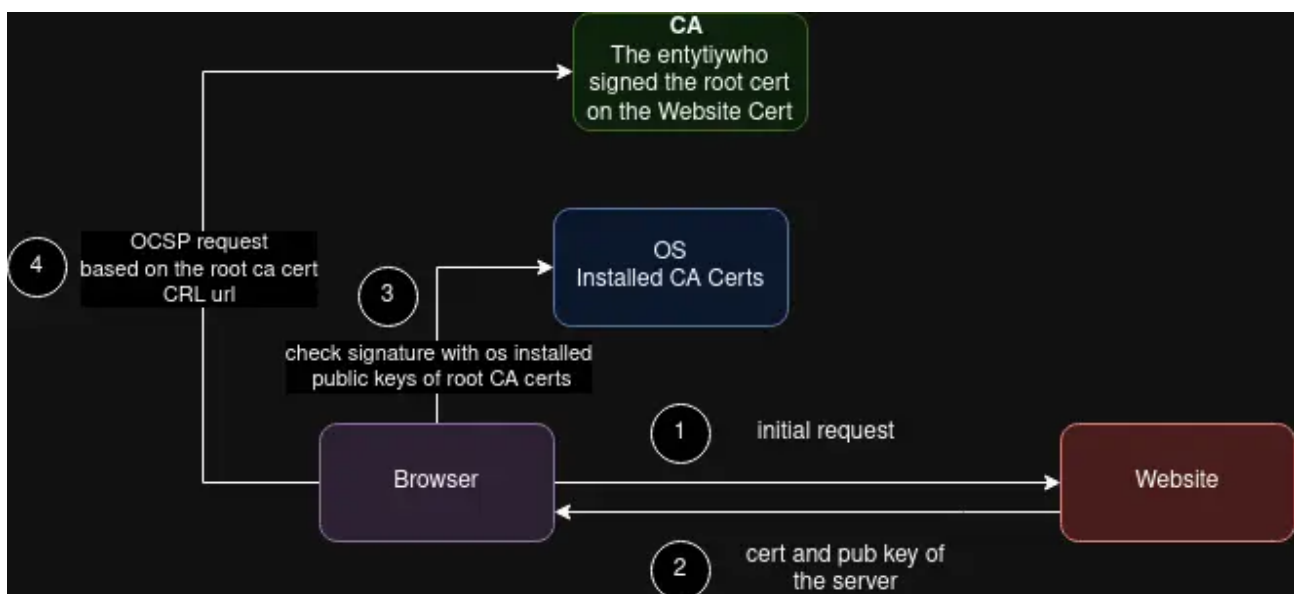
The hero of the day -> Certificate Authorities (Verification Authority)

A company that is considered trustworthy and produces digital certificates for other individuals and companies (i.e. subjects) bearing that subject's public key

Tasks

- issues digital certificates;
- helps establish trust between communicating entities over the internet
- verifies domain names and organizations to validate their identities
- maintains [certificate revocation lists \(CRL\)](#). 
- understands [Online Certificate Status Protocol \(OCSP\)](#) and responds to clients regarding the validity of the cert

How browsers validate the certificate of the website?



Browsers first checking the website cert against the os trusted root certs, then they are sending an OCSP request based on the certificate CRL Endpoint field in the certificate of the website.

Key Usages	
Purposes	Digital Signature
Extended Key Usages	
Purposes	Server Authentication
Subject Key ID	
Key ID	BD:AF:4D:02:7F:7E:9D:A5:6F:BB:CC:82:40:3E:19:18:D1:0C:3A:C6
Authority Key ID	
Key ID	90:77:92:35:67:C4:FF:A8:CC:A9:E6:7B:D9:80:79:7B:CC:93:F9:38
CRL Endpoints	
Distribution Point	http://c.pki.goog/we1/5XiTs4pnn2c.crl
Authority Info (AIA)	
Location	http://o.pki.goog/s/we1/dTI
Method	Online Certificate Status Protocol (OCSP)
Location	http://i.pki.goog/we1.crt
Method	CA Issuers
Certificate Policies	
Policy	Certificate Type (2.23.140.1.2.1)
Value	Domain Validation
Embedded SCTs	
Log ID	E6:D2:31:63:40:77:8C:C1:10:41:06:D7:71:B9:CE:C1:D2:40:F6:96:84:86:FB:BA:...
Signature Algorithm	SHA-256 ECDSA
Version	1
Timestamp	Sun, 17 Nov 2024 23:07:08 GMT
Log ID	4E:75:A3:27:5C:9A:10:C3:38:5B:6C:D4:DF:3F:52:EB:1D:F0:E0:8E:1B:8D:69:C0:...
Signature Algorithm	SHA-256 ECDSA
Version	1
Timestamp	Sun, 17 Nov 2024 23:07:07 GMT

How new certificates can be validated? -> Chain of trust

Hierarchy of certificates is used to verify the validity of a certificate's issuer. This hierarchy is known as a *chain of trust*. In a chain of trust, certificates are issued and signed by certificates that live higher up in the hierarchy. This way if the OS has some trusted root certificates pre installed it is easy to build a certificate hierarchy which can prove the validity of the leaf certificate.

Chain of trust tree main pillars

Trust Anchor

A public cert key named as CA cert which is trusted by both of the communicating parties, usually exists at OS level

Intermediate Cert

- you cannot issue new cert from trusted public CA certs since you need a new layer
- extra security to provide validation for the CA cert
- cheaper then issuing new CA cert for every need. New CA cert is expensive because it requires special validations of the issuer company and the processes [how this CA cert private keys are stored](#)

End Entity Cert / Leaf Cert

- no other cert can be above it
- represents the end entity of the chain

Cert Validation Levels

- Domain Validation -> the CA validates that the requestor really has the ownership of the domain in the CSR (Certificate Sign Request). Usually asking to put a file in a specific path under the domain, or putting a new DNS entry under the domain
- Organization Validation -> CA will contact the organization and will check every detail of the company that will be included in the cert
- Extended Validation -> CA will make full background check of the organization, by actually visiting the company and checking the IDs of the key employees etc.. This certificate type will print the company name in the browser bar above the certificate

Cert Validity Scopes

- Single Domain Certs -> for single domain (mydomain.com)
- Wildcard SSL Certs -> valid for any subdomain of one domain (*.mydomain.com)
- Multi-Domain SAN SSL Cert -> valid for multiple different domain (mydomain.com, mydomain2.com)
- Self signed certificates -> only valid on machines which has the signing CA cert installed in the OS trust store

Certificate Standards

| Ok but how does everyone understands the certs?

X.509 for the win! 🏆

This standard make possible to all participants in the PKI will understand the content of the certificate by enforcing that every information/property within a digital certificate are all placed in the same location and in the same order.

Parts 🏗️

As you can see above we can easily inspect a certificate with the openssl utility:

```
openssl x509 -in alice.crt -text -  
noout
```

Data Section

Data:

```
Version: 3 (0x2)
Serial Number: 1 (0x1)
Signature Algorithm: sha1WithRSAEncryption
Issuer: C=FR, ST=Alsace, L=Strasbourg, O=www.freelan.org, OU=freelan, CN=Freelan
Sample Certificate Authority, emailAddress=contact@freelan.org
Validity
  Not Before: Apr 27 10:31:18 2012 GMT
  Not After : Apr 25 10:31:18 2022 GMT
Subject: C=FR, ST=Alsace, O=www.freelan.org, OU=freelan, CN=alice,
emailAddress=contact@freelan.org
Subject Public Key Info:
  Public Key Algorithm: rsaEncryption
  Public-Key: (4096 bit)
  Modulus:
    00:dd:6d:bd:f8:80:fa:d7:de:1b:1f:a7:a3:2e:b2:
    02:e2:16:f6:52:0a:3c:bf:a6:42:f8:ca:dc:93:67:
    4d:60:c3:4f:8d:c3:8a:00:1b:f1:c4:4b:41:6a:69:
    d2:69:e5:3f:21:8e:c5:0b:f8:22:37:ad:b6:2c:4b:
    55:ff:7a:03:72:bb:9a:d3:ec:96:b9:56:9f:cb:19:
    99:c9:32:94:6f:8f:c6:52:06:9f:45:03:df:fd:e8:
    97:f6:ea:d6:ba:bb:48:2b:b5:e0:34:61:4d:52:36:
    0f:ab:87:52:25:03:cf:87:00:87:13:f2:ca:03:29:
    16:9d:90:57:46:b5:f4:0e:ae:17:c8:0a:4d:92:ed:
    08:a6:32:23:11:71:fe:f2:2c:44:d7:6c:07:f3:0b:
    7b:0c:4b:dd:3b:b4:f7:37:70:9f:51:b6:88:4e:5d:
    6a:05:7f:8d:9b:66:7a:ab:80:20:fe:ee:6b:97:c3:
    49:7d:78:3b:d5:99:97:03:75:ce:8f:bc:c5:be:9c:
    9a:a5:12:19:70:f9:a4:bd:96:27:ed:23:02:a7:c7:
    57:c9:71:cf:76:94:a2:21:62:f6:b8:1d:ca:88:ee:
    09:ad:46:2f:b7:61:b3:2c:15:13:86:9f:a5:35:26:
    5a:67:f4:37:c8:e6:80:01:49:0e:c7:ed:61:d3:cd:
    bc:e4:f8:be:3f:c9:4e:f8:7d:97:89:ce:12:bc:ca:
    b5:c6:d2:e0:d9:b3:68:3c:2e:4a:9d:b4:5f:b8:53:
    ee:50:3d:bf:dd:d4:a2:8a:b6:a0:27:ab:98:0c:b3:
    b2:58:90:e2:bc:a1:ad:ff:bd:8e:55:31:0f:00:bf:
    68:e9:3d:a9:19:9a:f0:6d:0b:a2:14:6a:c6:4c:c6:
    4e:bd:63:12:a5:0b:4d:97:eb:42:09:79:53:e2:65:
    aa:24:34:70:b8:c1:ab:23:80:e7:9c:6c:ed:dc:82:
    aa:37:04:b8:43:2a:3d:2a:a8:cc:20:fc:27:5d:90:
    26:58:f9:b7:14:e2:9e:e2:c1:70:73:97:e9:6b:02:
    8e:d3:52:59:7b:00:ec:61:30:f1:56:3f:9c:c1:7c:
    05:c5:b1:36:c8:18:85:cf:61:40:1f:07:e8:a7:06:
    87:df:9a:77:0b:a9:64:72:03:f6:93:fc:e0:02:59:
    c1:96:ec:c0:09:42:3e:30:a2:7f:1b:48:2f:fe:e0:
    21:8f:53:87:25:0d:cb:ea:49:f5:4a:9b:d0:e3:5f:
    ee:78:18:e5:ba:71:31:a9:04:98:0f:b1:ad:67:52:
    a0:f2:e3:9c:ab:6a:fe:58:84:84:dd:07:3d:32:94:
    05:16:45:15:96:59:a0:58:6c:18:0e:e3:77:66:c7:
    b3:f7:99
  Exponent: 65537 (0x10001)
X509v3 extensions:
  X509v3 Basic Constraints:
    CA:FALSE
  Netscape Comment:
    OpenSSL Generated Certificate
```

X509v3 Subject Key Identifier:

59:5F:C9:13:BA:1B:CC:B9:A8:41:4A:8A:49:79:6A:36:F6:7D:3E:D7

X509v3 Authority Key Identifier:

23:6C:2D:3D:3E:29:5D:78:B8:6C:3E:AA:E2:BB:2E:1E:6C:87:F2:53

Signature Section

- The cipher algorithm – the algorithm used by the issuer to create a digital signature
- The digital signature for the CA – a hash of all the information in the certificate encrypted with the CA private key

Signature Algorithm: sha1WithRSAEncryption

Signature Value:

13:e7:02:45:3e:a7:ab:bd:b8:da:e7:ef:74:88:ac:62:d5:dd:
10:56:d5:46:07:ec:fa:6a:80:0c:b9:62:be:aa:08:b4:be:0b:
eb:9a:ef:68:b7:69:6f:4d:20:92:9d:18:63:7a:23:f4:48:87:
6a:14:c3:91:98:1b:4e:08:59:3f:91:80:e9:f4:cf:fd:d5:bf:
af:4b:e4:bd:78:09:71:ac:d0:81:e5:53:9f:3e:ac:44:3e:9f:
f0:bf:5a:c1:70:4e:06:04:ef:dc:e8:77:05:a2:7d:c5:fa:80:
58:0a:c5:10:6d:90:ca:49:26:71:84:39:b7:9a:3e:e9:6f:ae:
c5:35:b6:5b:24:8c:c9:ef:41:c3:b1:17:b6:3b:4e:28:89:3c:
7e:87:a8:3a:a5:6d:dc:39:03:20:20:0b:c5:80:a3:79:13:1e:
f6:ec:ae:36:df:40:74:34:87:46:93:3b:a3:e0:a4:8c:2f:43:
4c:b2:54:80:71:76:78:d4:ea:12:28:d8:f2:e3:80:55:11:9b:
f4:65:dc:53:0e:b4:4c:e0:4c:09:b4:dc:a0:80:5c:e6:b5:3b:
95:d3:69:e4:52:3d:5b:61:86:02:e5:fd:0b:00:3a:fa:b3:45:
cc:c9:a3:64:f2:dc:25:59:89:58:0d:9e:6e:28:3a:55:45:50:
5f:88:67:2a:d2:e2:48:cc:8b:de:9a:1b:93:ae:87:e1:f2:90:
50:40:d9:0f:44:31:53:46:ad:62:4e:8d:48:86:19:77:fc:59:
75:91:79:35:59:1d:e3:4e:33:5b:e2:31:d7:ee:52:28:5f:0a:
70:a7:be:bb:1c:03:ca:1a:18:d0:f5:c1:5b:9c:73:04:b6:4a:
e8:46:52:58:76:d4:6a:e6:67:1c:0e:dc:13:d0:61:72:a0:92:
cb:05:97:47:1c:c1:c9:cf:41:7d:1f:b1:4d:93:6b:53:41:03:
21:2b:93:15:63:08:3e:2c:86:9e:7b:9f:3a:09:05:6a:7d:bb:
1c:a7:b7:af:96:08:cb:5b:df:07:fb:9c:f2:95:11:c0:82:81:
f6:1b:bf:5a:1e:58:cd:28:ca:7d:04:eb:aa:e9:29:c4:82:51:
2c:89:61:95:b6:ed:a5:86:7c:7c:48:1d:ec:54:96:47:79:ea:
fc:7f:f5:10:43:0a:9b:00:ef:8a:77:2e:f4:36:66:d2:6a:a6:
95:b6:9f:23:3b:12:e2:89:d5:a4:c1:2c:91:4e:cb:94:e8:3f:
22:0e:21:f9:b8:4a:81:5c:4c:63:ae:3d:05:b2:5c:5c:54:a7:
55:8f:98:25:55:c4:a6:90:bc:19:29:b1:14:d4:e2:b0:95:e4:
ff:89:71:61:be:8a:16:85

X509v3 extensions section

[X509 extensions](#) can be used to embed more metadata into the certificate itself. The ones which is the most interesting are:

- extendedKeyUsage -> This is used to determine the possible usage of the cert. So the programs using and validating can determine if it is sufficient for their use case
- subjectAltName -> this can include multiple domains names this way extending the original subject of the certificate. This is used for **SAN** certs.

PKCS#7 (Cryptographic Message Syntax Standard)

- PKCS7 is standard used to sign and encrypt messages particularly in email communications or to sign digital documents for example text files or pdfs etc..
- File extension: .p7b, .p7c, .p7m, .p7s
- The key is not bundled in the file
- One or more certificate packaged together but not signed or encrypted
- The signed document cannot be read without special tools like openssl

Create pkcs7 cert and key and sign and verify a document

generate a key

```
openssl genrsa -out private.key  
2048
```

generate a self signed cert

```
openssl req -new -x509 -key private.key -out certificate.crt -days 365 -subj  
"/C=US/ST=State/L=City/O=Organization/OU=Unit/CN=CommonName"
```

create the test message that we want to sign

```
echo "This is a test message." >  
message.txt
```

sign the document

```
openssl smime -sign -in message.txt -out signed_message.p7s -signer certificate.crt -inkey  
private.key -outform DER -nodetach
```

print cert from signed document

```
openssl pkcs7 -in signed_message.p7s -print_certs -noout
```

verify the document with

```
openssl smime -verify -in signed_message.p7s -CAfile certificate.crt -inform DER
```

```
openssl smime -sign -in message.txt -out signed_message.p7s -signer certificate.crt -inkey private.key -outform DER -nodetach
```

Certificate formats

Certificate format is the way how the certificate is encoded, stored and which file extension is used.

Binary encoded

.DER (Distinguished Encoding Rules)

- File extensions: .der, .cer
- It is not a bundle format so either the key or the cert is stored in it
- Often used by Java applications running as web servers
- There are no human readable headers in the file

How to create and read it?

Generate an rsa key

```
openssl genrsa -out private.key 2048
```

generate a self signed cert with this key

```
openssl req -new -x509 -key private.key -out certificate.crt -days 365
```

Convert private key to der format

```
openssl rsa -in private.key -outform DER -out  
private.der
```

Convert cert to der format

```
openssl x509 -in certificate.crt -outform DER -out  
certificate.der
```

Read the created cert

```
openssl x509 -in certificate.der -inform DER -text -  
noout
```

Read the created key

```
openssl rsa -in private.der -inform DER -text -  
noout
```

PKCS#12 (Public-Key Cryptographic Standards) sometimes references as PFX

- File extensions: .pfx, .p12
- This file is encrypted and protected with a password
- The pfx file contain multiple cryptographic objects including public key certificates, private keys, intermediate keys.
- **One or more certificates packed together, password-encrypted**
- Used when someone wants to store and transfer private keys and cert bundles securely together in one file

How to create and read it?

generate the private key

```
openssl genrsa -out private.key  
2048
```

create a self signed certificate with you new key

```
openssl req -new -x509 -key private.key -out certificate.crt -days  
365
```

create the pfx file

```
openssl pkcs12 -export -out certificate.pfx -inkey private.key -in  
certificate.crt
```

As you noticed this export asked for a password. This will be needed when we want to read the pfx like this:

```
openssl pkcs12 -info -in  
input.pfx
```

This will output something like this:

```

> openssl pkcs12 -info -in certificate.pfx
Enter Import Password:
MAC: sha256, Iteration 2048
MAC length: 32, salt length: 8
PKCS7 Encrypted data: PBES2, PBKDF2, AES-256-CBC, Iteration 2048, PRF
hmacWithSHA256
Certificate bag
Bag Attributes
    localKeyID: 69 0A CA 69 19 B8 23 24 CD DE B9 F8 D1 C8 CA 19 F0 63 68 F5
subject=C=AU, ST=Some-State, O=Internet Widgits Pty Ltd
issuer=C=AU, ST=Some-State, O=Internet Widgits Pty Ltd
-----BEGIN CERTIFICATE-----
MIIDazCCA0gAwIBAgIUCz7AnQ57KFLQTEhMv+2H2c1u7vwvDQYJKoZIhvcNAQEL
BQAwRTElMAkGA1UEBhMCQVUxEzARBgNVBAGMClNvbWUtU3RhdGUxITAfBgNVBAoM
GEludGVybWV0IFdpZGdpdHMgUHR5IEEx0ZDAeFw0yNDEzMjIxMDIzNDBaFw0yNTEy
MjIxMDIzNDBaMEUxCzAJBgNVBAYTAkFVMRMwEQYDQIDApTb211LVN0YXRlMSEw
HwYDVQQKDBhJbnRlcm5ldCBXaWRnaXRzIFB0eSBMdGQwggEiMA0GCSqGSIb3DQEB
AQUAA4IBDwAwggEKAoIBAQCXxyB1Pl7U55vnwgS2qcjrfG0sRaP7L3d3/NTD+bx3
i0YXLA91oDaBM3aBtJ0FCTmhA/dAkeBfnxIA0nCuNGR2B+Iy0kXipZlk/825HCnP
1QtFlRxp07Hdq8+JqjGG+/0b3DghB3/0hH/xuGUD22cQmgJAL9ue6cIYx2Kp8Lxp
tqsFBQ/1EDHoJRFr3SyLyxWcPVgoGucYzAmifsGIMD2/8blNuynDTm0SdAJX6/XQ
Z7QzCqWfJQ6R03oCWsIVg10+oNPLYvbehho0Up0wRJoAWFS4476A9+n9iD9qQIUc
S1hHkkYw5UTzSCbsve1nIBRSHgbCMgJYNkb4GN8e8caLAgMBAAGjUzBRMB0GA1Ud
DgQWBBRJjaQ1lt9rWS4+Lfi9v+a21GLWHjAfBgNVHSMEGDAWgBRJjaQ1lt9rWS4+
Lfi9v+a21GLWHjAPBgNVHRMBAf8EBTADAQH/MA0GCSqGSIb3DQEBwUAA4IBAQBb
jGYjULouJmqdYP6GIhFoZP4JrenU9yRnZiFW14d2uDMlCwgTXych/4ibL6S4Pa6X
NPLDL71WCC5l90R26cnsZzI5Ns/4jt7IXdAhhQl7yLe9kWfXxHBTlhiONb6UkjR
JVnj2SUG2NpNHHN7+ZE/uScPexHRhuD0gcEnx7QUQvLQwrUy+ck9dkwV+EsIWhZR
7kBgVanfQ4nKmkdr2a2nGbcZk4SGsSx5R56xYwXuiuphA0j9TUBi7IGDhbvfpBQt
uw7YyIkDrHDxnCaF2jVVJARANSOj5/7oB0GIKMdLVdzomtJCQua0RdAjRT5iKPYO
TocjY9zu+z0S7740gGbU
-----END CERTIFICATE-----
PKCS7 Data
Shrouded Keybag: PBES2, PBKDF2, AES-256-CBC, Iteration 2048, PRF hmacWithSHA256
Bag Attributes
    localKeyID: 69 0A CA 69 19 B8 23 24 CD DE B9 F8 D1 C8 CA 19 F0 63 68 F5
Key Attributes: <No Attributes>
Enter PEM pass phrase:
Verifying - Enter PEM pass phrase:
-----BEGIN ENCRYPTED PRIVATE KEY-----
MIIFNTBfBgkqhkiG9w0BBQ0wUjAxBgkqhkiG9w0BBQwwJAQQmh45yx3+0oIeYwG0
yVMEgwICCAAwDAYIKoZIhvcNAgkFADAdBgglghkgBZQMEASoEE0Q78AVEq+EcPEW9
osl+xxMEggTQLSg2nLT7sIjvU7I0Embkg0XviYhLIGS6kaRis14aMAvJayTcv3C0
nSHlf6mxnkvfvt6wKgNui3x7qAXtubxQK0MXG9nXt8VHfoHUII1J3h/URLBIRHyv
R5PuNZDdQ1dEqUcrVzTIt4wrP82Nx9yPD3ICS+tJaNZTJPYvrABofljaXFJth3U
EIGriuUM3qpcpfFhkGduiJDijxRYcI95QZERcDJ/VpX2ZEa5ad+YdYRT9t12VbhV
lQhWvT5gcM/3GDuRkVdveC0qTlxwNFmy0Gus3R01Z/cvDwVRd8SvNIYPjKiB7MuW
sg0l1VayRhN81TUCA8dgyvdsDjrMx8X3WH4vqxNZ5BJymRfkji75VhC0Tm/OifDh
Vw09tTn/U/MFIZggK5IuaZl9y9fbmiYCFStMxz6euBj7eXyT4xJTbuzNURwQ3R2
S4a0bz/tj5H7aIP4qpVCLTSRaXbuB/2h70HMg0Jal6JhPKAZDKFiPhPQF0ll1D2E
eQP8iS7X6k6HgDixJ0PUB6mlqBwVEIN00oekPaWpie7zgyNZIEw4U4Z6dilmLcTt
RX+0smZ7T+j8mNx0aQLboIOnoNVLgiNgZCybl1ZFtkHBGBS51giz7Gaen+W4HjLz
/Gv66/HHXK1oJxtJvUyiSxNVWtu/AwGwv99u5ahC97Ct/C7nTDVF7hj2Q2RMYdxN
0VnK100kp+7cKlp8aHAKKdhlp1JswMcFv+a0tpNWgF0yvsWQUJGZ8BI9qLDZ5
iuNn5EUlw7PsS+7XPDlVarjYjzBYCEwG4E4ukxPkFw/bWVvdtAd0IsiBvx91hTT/
vCkF92lm/lckYARaucSZ0PRGD9ozbhw/6xcImCF6ME764VxkhESvLDNqLoAY4MUa
5GSLLnDdhzPkuYLqnzoZiKiBC1Y4QZ47KsRnZtiFJwCLXuxU982qT7bxVEIWRHt9
ER0iSHSVcNsr7ThBk1o25cDpyd5q0+VANzrBZ+yz1R0iifwiFgSLPLK0rlyRQM3
oSHdtUYVqe3ZF7wdEnGV3c4UfpyDrU1EWxv4JK44Kx6fMS+T/gvsW4HpNG9a0wa9

```

cMbTZ0i8z+amiFu402vGFo7fa+3R1ztb900aasDtqgty11tcW+bN0oTp+fNp7d/M
WsINwe/W2UJSkQvnEHkgs0GytmGAL8400D5Zbqv9tzBzzpn9wzoGI6jec58epzv
uRhnbJ0cqDvP0V2NhXx1uGUP44A0eR5BK/mxj22m+uh+5IW9Kfk+8q5fHXQkDkKp
qqXS5K6kjLp8WZurdeWFHevoEZqaSF86lGSXMNn2Zv98oJmSrnxV90h4qB465zm5
ya4vJHX0+yVIXfFRpBqCaRcTB4BGZj33fIxp+baAZM0TIs14Y82lFcW7JojrNa0p
kxsAKE2GUukVdogysS5oGAtUbnB8o0ckGBRt1UZ+M5AHgWkrCKQHK18V0K0QFw6V
sgey0orZpqBCCWuvVSR2n9KsBWyUtvo5Sb9xUt2YERi5EVctWHfCFRhFXP3u6qX/
3WKZySiYAPrerUuAQe+wG3esjxPXf9qv1uKLmRC7pUD/raACbECE63nKBS1ltMss
1uUl+6lCnlingASa7hbJ04UK11V4DW8ukDCkR95cu8cKP7nGm1mc+Fc=
-----END ENCRYPTED PRIVATE KEY-----

As you can see it really contains the cert and the encrypted private key. This is a safe format to store and share your certificate bundles

Base64 (ASCII)

.PEM (Privacy-Enhanced Mail)

- File extensions: .pem, .crt, .cer, .key
- **There is no encryption to protect the keys so the owner has to transfer them securely**
- Default format for OpenSSL. Suitable for sending files as text between systems
- Can contain multiple certs in one file by simply concatenating the key and cert
- Certificates are surrounded with:
 - —BEGIN CERTIFICATE—
 - —END CERTIFICATE—
- Keys are surrounded with:
 - —BEGIN RSA PRIVATE KEY—
 - —END RSA PRIVATE KEY—
- There are cases when more than one cert / key has to be included in the file like when the server does SSL termination and other intermediate cert is required to establish the full chain of trust:

```
-----BEGIN RSA PRIVATE KEY-----
... (private key data) ...
-----END RSA PRIVATE KEY-----
-----BEGIN CERTIFICATE-----
... (server certificate data) ...
-----END CERTIFICATE-----
-----BEGIN CERTIFICATE-----
... (intermediate certificate data)
...
-----END CERTIFICATE-----
-----BEGIN CERTIFICATE-----
... (root certificate data) ...
-----END CERTIFICATE-----
```

How to create and read them:

generate key

```
openssl genrsa -out private.key  
2048
```

generate a self signed cert

```
openssl req -new -x509 -key private.key -out certificate.pem -days  
365
```

combine them in one file

```
cat private.key certificate.pem >  
combined.pem
```

Reading only the key from this combined file

```
openssl rsa -in combined.pem -text -noout
```

Reading all the certs from this combined file

```
openssl x509 -in combined.pem -text -  
noout
```

How to get a new certificate?

Option A: Asking the the big brother (CA) with a love letter (CSR) to create one for you!

- you should decide the certificate type
- select a Certificate Authority like
 - DigiCert
 - GlobalSign
 - Sectigo
 - GoDaddy
 - etc..

- the CA will ask for a proof of domain ownership
 - email validation:
The CA will send a verification email message to an email address under your domain
 - DNS validation:
CA will ask you to put a new TXT entry under your domain name with a specific value
 - HTTP validation:
CA will ask you to upload a file to a specific path which can be reached through your domain like domain/.wellknown/pki-validation/
- Send the CSR file and present the necessary extra documentation for EV/OV validation like utility bills, public records etc..
- CA will issue your new certificate and send it back to you

Example to create and decode CSR

(Useful online tool to decode CSRs) <https://ssltools.godaddy.com/views/csrDecoder>

generate csr

```
openssl req -newkey rsa:2048 -keyout example.key -out  
example.csr
```

This will create a key and a csr. Lets see the csr:

```

-----BEGIN CERTIFICATE REQUEST-----
MIIC9DCCAdwCAQAwfTElMAkGA1UEBhMCSFUxDjAMBgNVBAgMBUZlamVyMRAwDgYD
VQQHDAZwZWxlbmNlMRAwDgYDVQQKDAUZWwla29tMRkwFwYDVQQDDBB0ZXN0LmV4
YW1wbGUuY29tMR8wHQYJKoZIhvcNAQkBFhB0ZXN0QGV4YW1wbGUuY29tMIIBIjAN
BgkqhkiG9w0BAQEFAA0CAQ8AMIIBCgKCAQEAn7WtQNMvPgWGRkQk3NyZpv+HHRL/
5mxHluJ1u5v2V+1Q+zP0ctKFmksOP/hlSlj7v7+QWYuqWLLA4vqCx43lzLN8f6pB
WHtefYXKBHm8cT05QvAPXfeWmulGTsKESB+SETiAzjuep6+kVmslomogQnkk0sr8
xAye/XLrdPskN1IKLjIHpSi0+eR2oyr1PfJoBvin5qZDTg6DP43N5MpaCyF0GwTB
9PPprtHo8jJ0u9j60Brv7ED7VkygVbwpNEfMznjDaSKKCPB/8Xlvn9oxc182956U
4Vc5VKThCe9NE3PC0b2a2DDGXA0ipY1r+l/fFamNI9TzJaFP0fCnmYtXhwIDAQAB
oDIwEwYJKoZIhvcNAQkHMqYMBHRlc3QwGwYJKoZIhvcNAQkCMQ4MDHRlc3QgY29t
cGFueTANBgkqhkiG9w0BAQsFAA0CAQEAWAXxLyInas6xsVExlSi8w7Pkl238oHNc
30j9MJNVw1iiBMMgL9WBrIQ8g/Qn64eJi+IMKz0oan+2ajf0ZhsRziuS5u24PuvP
TNNqq2hr5Aq0AnzGZyHP0hqwIJpm203gQh6/4GpphxwEbr4lzGtv3rzXl2DjZMyw
0fXQsxBlys20/y75amB3qrNM1WhFNnC4o+auIsocDckXuBvjx2o0zx3MJxt4J+V9
fRSzdCqWAnkCyePaxvoi09l/YRReG7Cfi59PCo4DnVcW0qvSyutJ5em4IBxDCMH8
SZKQBrGKRkFfM1ax7UDCC0vCghEBW5Wg56r1HD89zXeZVzu1nQyZFg==
-----END CERTIFICATE REQUEST-----

```

Nothing too fancy right?

Lets dissect it:

```

openssl req -in example.csr -noout -
text

```

Certificate Request:

Data:

Version: 1 (0x0)

Subject: C=HU, ST=Fejer, L=Velence, O=Telekom, CN=test.example.com,
emailAddress=test@example.com

Subject Public Key Info:

Public Key Algorithm: rsaEncryption

Public-Key: (2048 bit)

Modulus:

00:9f:b5:ad:40:d3:2f:3e:05:86:46:44:24:dc:dc:
99:a6:ff:87:1d:12:ff:e6:6c:47:96:e2:75:bb:9b:
f6:57:ed:50:fb:33:f4:72:d2:85:9a:4b:0e:3f:f8:
65:4a:58:fb:bf:bf:90:59:8b:aa:58:b9:40:e2:fa:
82:c7:8d:e5:cc:b3:7c:7f:aa:41:58:7b:5e:7d:85:
ca:04:79:bc:71:3d:39:42:f0:0f:5d:f7:96:9a:e9:
46:4e:c2:84:48:1f:92:11:38:80:ce:3b:9e:a7:af:
a4:56:6b:25:a2:6a:20:42:79:24:d2:ca:fc:c4:0c:
9e:fd:79:6b:74:f4:a4:37:52:0a:96:32:07:a5:28:
b4:f9:e4:76:a3:2a:f5:3d:f2:68:06:f8:a7:e6:a6:
43:4e:0e:83:3f:8d:cd:e4:ca:5a:0b:21:4e:1b:04:
c1:f4:f3:e9:ae:d1:e8:f2:32:74:bb:d8:fa:d0:1a:
ef:ec:40:fb:56:4c:a0:55:bc:29:34:47:cc:ce:78:
c3:69:22:8a:0a:90:7f:f1:79:6f:9f:da:31:73:5f:
36:f7:9e:94:e1:57:39:54:a4:e1:09:ef:4d:13:73:
c2:39:bd:9a:d8:30:c6:5c:03:a2:a5:8d:6b:fa:5f:
df:14:09:8d:23:d4:f3:25:a1:4f:d1:f0:a7:99:8b:
57:87

Exponent: 65537 (0x10001)

Attributes:

challengePassword :test

unstructuredName :test company

Requested Extensions:

Signature Algorithm: sha256WithRSAEncryption

Signature Value:

58:05:f1:2f:22:27:6a:ce:b1:b1:51:31:95:28:bc:c3:b3:e4:
97:6d:fc:a0:73:5c:dc:e8:fd:30:93:55:c3:58:a2:04:c3:20:
2f:d5:81:ac:84:3c:83:f4:27:eb:87:89:8b:e2:0c:2b:3d:28:
6a:7f:b6:6a:37:ce:66:1b:11:ce:2b:92:e6:ed:b8:3e:eb:cf:
4c:d3:6a:ab:68:6b:e4:0a:8e:02:7c:c6:67:21:cf:d2:1a:b0:
20:9a:66:d8:ed:e0:42:1e:bf:e0:6a:69:87:1c:04:6e:be:35:
cc:6b:6f:de:bc:d7:97:60:e3:64:cc:b0:d1:f5:d0:b3:10:65:
ca:cd:8e:ff:2e:f9:6a:60:77:aa:b3:4c:d5:68:45:36:70:b8:
a3:e6:ae:22:ca:1c:0d:c9:17:b8:1b:e3:c7:6a:0e:cf:1d:cc:
27:1b:78:27:e5:7d:7d:14:b3:74:2a:96:02:79:02:c9:e3:da:
c6:fa:22:3b:d9:7f:61:14:5e:1b:b0:9f:8b:9f:4f:0a:8e:03:
9d:57:16:3a:ab:d2:ca:eb:49:e5:e9:b8:20:1c:43:08:c1:fc:
49:92:90:06:b1:8a:46:41:5f:33:56:b1:ed:40:c2:08:eb:c2:
82:11:01:5b:95:a0:e7:aa:f5:1c:3f:3d:cd:77:99:57:3b:b5:
9d:0c:99:16

You can see there is a signature with the private key and all the properties that can be validated by the CA. The CA will validate these data during the process. This validation is different based on the required validation scope and level.

Option B: Self signed

We have already created some self signed certificate above, but now we will create it a way that the OS will trust it! To achieve this we have to create a CA cert also and add it to the OS trusted root cert store, then create a CSR which will be signed by our root CA cert.

Create key for CA cert

```
openssl genrsa -out rootCA.key  
2048
```

Create root certificate

```
openssl req -x509 -new -nodes -key rootCA.key -sha256 -days 3650 -out  
rootCA.crt
```

Add to the OS trusted root cert store (linux)

```
sudo cp rootCA.crt /usr/local/share/ca-  
certificates/
```

Apply it

```
sudo update-ca-  
certificates
```

Okay now create our server key

```
openssl genrsa -out server.key  
2048
```

Now we have to create a CSR which will be signed by the CA cert later

```
openssl req -new -key server.key -out  
server.csr
```

Time to sign the CSR with CA cert

```
openssl x509 -req -in server.csr -CA rootCA.crt -CAkey rootCA.key -CAcreateserial -out  
server.crt -days 365 -sha256
```

Try it out!

start a test server with openssl (amazing right? It is capable of this)

```
openssl s_server -cert server.crt -key server.key -CAfile rootCA.crt -accept  
4433
```

In a new terminal try to connect it

```
openssl s_client -connect localhost:4433 -CAfile  
rootCA.crt
```

With this commands you can deeply debug your certificate and the TLS session itself

```

> openssl s_server -cert server.crt -key server.key -CAfile rootCA.crt -accept 4433
Using default temp DH parameters
ACCEPT
-----BEGIN SSL SESSION PARAMETERS-----
MIGDAgEBAgIDBAQCEwIEIMgh++wyAnXLISZyURB2+Ri9yzg2IkzQZioUBDQxqtjj
BDD+ACxT0fWiGktJAXTzaxG/3YsvhkxB0SQGbjZU9r5/PvLMrJ0M/KXQHZrwz3Am
wcWhBgIEZ0GQRKIEAgIcIKQGBAQBAAAArgcCBQC+0VG1swMCAR0=
-----END SSL SESSION PARAMETERS-----
Shared
ciphers:TLS_AES_256_GCM_SHA384:TLS_CHACHA20_POLY1305_SHA256:TLS_AES_128_GCM_SHA256:ECDHE-
ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:DHE-RSA-AES256-GCM-SHA384:ECDHE-ECDSA-
CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305:DHE-RSA-CHACHA20-POLY1305:ECDHE-ECDSA-AES128-
GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:DHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-AES256-
SHA384:ECDHE-RSA-AES256-SHA384:DHE-RSA-AES256-SHA256:ECDHE-ECDSA-AES128-SHA256:ECDHE-RSA-
AES128-SHA256:DHE-RSA-AES128-SHA256:ECDHE-ECDSA-AES256-SHA:ECDHE-RSA-AES256-SHA:DHE-RSA-
AES256-SHA:ECDHE-ECDSA-AES128-SHA:ECDHE-RSA-AES128-SHA:DHE-RSA-AES128-SHA:AES256-GCM-
SHA384:AES128-GCM-SHA256:AES256-SHA256:AES128-SHA256:AES256-SHA:AES128-SHA
Signature Algorithms:
ECDSA+SHA256:ECDSA+SHA384:ECDSA+SHA512:Ed25519:Ed448:ECDSA+SHA256:ECDSA+SHA384:ECDSA+SHA512
:RSA-PSS+SHA256:RSA-PSS+SHA384:RSA-PSS+SHA512:RSA-PSS+SHA256:RSA-PSS+SHA384:RSA-
PSS+SHA512:RSA+SHA256:RSA+SHA384:RSA+SHA512:ECDSA+SHA224:RSA+SHA224:DSA+SHA224:DSA+SHA256:D
SA+SHA384:DSA+SHA512
Shared Signature Algorithms:
ECDSA+SHA256:ECDSA+SHA384:ECDSA+SHA512:Ed25519:Ed448:ECDSA+SHA256:ECDSA+SHA384:ECDSA+SHA512
:RSA-PSS+SHA256:RSA-PSS+SHA384:RSA-PSS+SHA512:RSA-PSS+SHA256:RSA-PSS+SHA384:RSA-
PSS+SHA512:RSA+SHA256:RSA+SHA384:RSA+SHA512:ECDSA+SHA224:RSA+SHA224
Supported groups:
x25519:secp256r1:x448:secp521r1:secp384r1:ffdhe2048:ffdhe3072:ffdhe4096:ffdhe6144:ffdhe8192
Shared groups:
x25519:secp256r1:x448:secp521r1:secp384r1:ffdhe2048:ffdhe3072:ffdhe4096:ffdhe6144:ffdhe8192
CIPHER is TLS_AES_256_GCM_SHA384

```

```

> openssl s_client -connect localhost:4433 -CAfile rootCA.crt
Connecting to 127.0.0.1
CONNECTED(00000009)
Can't use SSL_get_servername
depth=1 C=AU, ST=Some-State, O=Internet Widgits Pty Ltd
verify return:1
depth=0 C=AU, ST=Some-State, O=Internet Widgits Pty Ltd
verify return:1
---
Certificate chain
 0 s:C=AU, ST=Some-State, O=Internet Widgits Pty Ltd
  i:C=AU, ST=Some-State, O=Internet Widgits Pty Ltd
  a:PKEY: rsaEncryption, 2048 (bit); sigalg: RSA-SHA256
  v:NotBefore: Nov 23 08:10:23 2024 GMT; NotAfter: Nov 23 08:10:23 2025 GMT
 1 s:C=AU, ST=Some-State, O=Internet Widgits Pty Ltd
  i:C=AU, ST=Some-State, O=Internet Widgits Pty Ltd
  a:PKEY: rsaEncryption, 2048 (bit); sigalg: RSA-SHA256
  v:NotBefore: Nov 23 08:09:36 2024 GMT; NotAfter: Nov 21 08:09:36 2034 GMT
---
Server certificate
-----BEGIN CERTIFICATE-----
MIIDWjCCAKKgAwIBAgIUWCSl0d3FwC1M523YmR3NJGIcrC4wDQYJKoZIhvcNAQEL
BQAwRTElMAkGA1UEBhMCQVUxEzARBgNVBAGMClNvbWUuU3RhdGUxITAfBgNVBAoM
GEludGVybWV0IFdpZGdpdHMgUHR5IEEx0ZDAeFw0yNDEwMjMwODEwMjNaFw0yNTEw
MjMwODEwMjNaMEUxChAJBgNVBAYTAFVMRMwEQYDVQIDApTb211LVN0YXRlMSEw
HwYDVQQKBDBhJbnRlcmlzcm5ldCBXaWRnaXRzIFB0eSBMdGQwggEiMA0GCSqGSIb3DQEB
AQUAA4IBDwAwggEKAoIBAQC1UK6+tWhKWumwAEYPNrk3Mi4Jy+gCrL4RjAVRsLaj
zbSNEfDKubsI1LE6v3w9P1a670mmY0AXfPWh6rN9xKvJ9EP0HHLnKi48XUYfI+Eu
hhIuCdY0FYbYGcfGuqyHsYCUfi52NB3l4lUGexLGBnFtw0b/xgZclsJf209v9Bln
WXRrJx7SM+9gKafA0GJuLGGjH0S0d+uikirbZWduKctoJ9jjJ9tGoF0a0XucyiFz
l3Ems2QD6wHS+vYyvSkvVCLXamZdtJ+g69wvf+293Bmj0tSZyK4ZnnSBFEK9tVYH
zRrPWTrJw8j4clNvbq9R18WE2q0Z3iz+YNMr0JKI89f3AgMBAAGjQjBAMB0GA1Ud
DgQWBRLikeIL8RofsaS1+ee4+920eF25DAfBgNVHSMEGDAwBT54cF8S9EV5E0I
W21aNrPv3U8K4jANBgkqhkiG9w0BAQsFAA0CAQEAGTY1+3ijMjd10b0C4Vau+uMI
KfLwFxxzQJ0w45+M0RKg9zow+aGnoWioAqdytVkXnQNFkqBrHJ6brXxy35LmCNRA
pGAF/SDZCGbiVQqPyKlHkFrEYzP9oCBFveCle2wVOD7zpbNDbpbSt5kC4c68sa
de5Lp0u4Pt547zenngPI7GyQCdB3bz7XiyyuaEk6CRb8+uz00Ixxv0dcWqu2IIDA
5aI/97W/JCveEKgDcbny8vpV/oZwm+AlxUmp5NS33z+0hw4XlPj9+9B1/LLiX/m5
7K8fdB0o+QM/ieDNwWe18bNe5jkQt0UARJFCHxUebVcYeSXHDk9sdb/stkGb/g==
-----END CERTIFICATE-----
subject=C=AU, ST=Some-State, O=Internet Widgits Pty Ltd
issuer=C=AU, ST=Some-State, O=Internet Widgits Pty Ltd
---
No client certificate CA names sent
Peer signing digest: SHA256
Peer signature type: RSA-PSS
Server Temp Key: X25519, 253 bits
---
SSL handshake has read 2302 bytes and written 382 bytes
Verification: OK
---
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
Protocol: TLSv1.3
Server public key is 2048 bit
This TLS version forbids renegotiation.
Compression: NONE
Expansion: NONE
No ALPN negotiated
Early data was not sent

```

Verify return code: 0 (ok)

Post-Handshake New Session Ticket arrived:

SSL-Session:

Protocol : TLSv1.3

Cipher : TLS_AES_256_GCM_SHA384

Session-ID: 76B8EBB15D24AA9A7A41DD6BC1CA6DF77D89F4458C6517A974B54DCA2E9D0AE0

Session-ID-ctx:

Resumption PSK:

ECB7546E2D711C3BA419C0F888AE2B39043BE65324E1C9E485F573D38F85B42E20AEBB2B0EBDD9B106B6BFABB283CC24

PSK identity: None

PSK identity hint: None

SRP username: None

TLS session ticket lifetime hint: 7200 (seconds)

TLS session ticket:

0000 - 08 72 9b 9c 71 22 2e 4a-04 4a a6 8d e9 51 b8 3e	.r..q".J.J...Q.>
0010 - 4b d4 de 72 53 55 2d 25-04 fa 2f 15 33 b9 46 1c	K..rSU-%/.3.F.
0020 - 2d 16 13 4b 32 c2 b3 46-ff 16 76 c7 0e 28 ea 2d	...K2..F..v..(.-
0030 - 70 e3 95 80 bc dd af 40-e6 05 47 67 51 9e b1 0a	p.....@..GgQ...
0040 - 6b 6e a7 15 53 ba 07 8b-ec 92 3b d1 8c cc 83 77	kn..S.....;....w
0050 - 5f 83 33 ae 10 7b f4 94-0c 25 a9 e3 e6 b1 bc d7	_.3..{...%.....
0060 - fa 33 5b 49 e3 b8 a4 d9-0a fb ef 16 a2 5b 75 b3	.3[I.....[u.
0070 - 33 6e d4 bf ec d3 ab 2a-0f 50 fd 94 a1 37 e2 24	3n.....*.P...7.\$
0080 - 85 da 2a 7b f2 80 40 0c-6d a9 7d 4c e1 c6 d2 80	..*{..@.m.}L....
0090 - 9c 06 5f 36 76 04 11 61-67 df 74 7f e9 e6 27 cc	.._6v..ag.t...'
00a0 - 99 07 db ee 1e 1b a6 84-c8 37 87 87 01 40 45 cd	7...@E.
00b0 - 90 6a 40 97 2b df 17 ac-d5 da c3 5e 74 b5 2f ca	.j@.+.....^t./.
00c0 - bf 12 29 be a7 1d c2 af-16 54 a9 df 40 1d 28 4d	..).....T..@.(M

Start Time: 1732350020

Timeout : 7200 (sec)

Verify return code: 0 (ok)

Extended master secret: no

Max Early Data: 0

read R BLOCK

Post-Handshake New Session Ticket arrived:

SSL-Session:

Protocol : TLSv1.3

Cipher : TLS_AES_256_GCM_SHA384

Session-ID: F956D4DC78F7CEE8859EE8DE6B6A3D608E8283D08141CEE9D98635F45F12B5B

Session-ID-ctx:

Resumption PSK:

FE002C5339F5A21A4B490174F36B11BFDD8B2F864C41D124066E3654F6BE7F3EF2CCAC9D0CFC5A5D01D9AF0CF7026C1C5

PSK identity: None

PSK identity hint: None

SRP username: None

TLS session ticket lifetime hint: 7200 (seconds)

TLS session ticket:

0000 - 08 72 9b 9c 71 22 2e 4a-04 4a a6 8d e9 51 b8 3e	.r..q".J.J...Q.>
0010 - db c6 5e 32 62 28 57 54-6a 95 e5 34 df 9a 6c f6	..^2b(WTj...4..l.
0020 - 7c 69 d5 82 d8 89 fd 77-7d 64 70 61 d7 bf ed 18	i.....w}dpa....
0030 - f2 6b 16 67 f7 4e 42 3a-84 13 7b 9a d7 09 9f a2	.k.g.NB:...{.....
0040 - 17 79 e9 74 89 fb 1c 06-a8 a1 8e 91 eb e1 91 59	.y.t.....Y
0050 - 1b b5 77 7a 51 8d 4c 55-98 88 da c9 5e 15 65 09	..wzQ.LU....^..e.
0060 - 37 20 9c b9 36 9b 9c de-2c 94 32 a5 34 21 17 2c	7 ..6....,2.4!.,

```
0070 - 1c e2 7b 76 fc f3 74 4d-d4 d0 6b e8 7a b9 83 07 ..{v...tM..k.z...
0080 - 1a 28 d2 9c 66 96 09 a4-a5 57 30 5f b3 c1 67 b5 .(..f....W0_..g.
0090 - f7 8d 9e ed 4a 0e 00 36-68 ac 0e 40 7b cd 13 38 ....J..6h..@{..8
00a0 - 9a 70 96 f1 c6 b9 bf 3f-ff ee 19 8a fa 16 f9 d1 .p.....?.....
00b0 - 8b 74 18 2a ee 7a 23 44-3b 2b cc 79 f2 1f 06 04 .t.*.z#D;+.y....
00c0 - 31 55 08 eb 52 34 ef 7a-4c ea 09 4c 8b 73 dd c2 1U..R4.zL..L.s..
```

```
Start Time: 1732350020
Timeout    : 7200 (sec)
Verify return code: 0 (ok)
Extended master secret: no
Max Early Data: 0
```

```
---
read R BLOCK
```


Option C: Let's Encrypt free Certificate Authority

Let's Encrypt is a free, automated, and open Certificate Authority (CA) that provides SSL/TLS certificates to secure websites using ACME protocol. It simplifies the process of obtaining, installing, and renewing certificates using automation tools like Certbot.

Pros

- its free
- easy to automate without human intervention
- backend by nonprofit company
- trusted by all major browsers and devices

Cons

- issued certs lifetime is only 90 days
- only domain validated certs are possible (no EV or OV)
- no warranty

What is ACME: Automatic Certificate Management Environment. It is a protocol designed to automate the process of issuing, renewing, and revoking SSL/TLS certificates. ACME eliminates the need for manual certificate management by allowing software to interact with a Certificate Authority (CA) directly.

Validation methods

- HTTP-01 Challenge: Place a specific file in a `.well-known/acme-challenge/` directory on your server.
- DNS-01 Challenge: Add a TXT record to your domain's DNS configuration.
- TLS-ALPN-01 Challenge: Requires the server to support ALPN (Application-Layer Protocol Negotiation) and respond to a challenge over HTTPS.

Suggested usage with DNS validation

I prefer using this method since it only requires that you delegate your domain ownership from the original domain registrar to CloudFlare and use the CloudFlare API with some automation to renew your certificate.

The process itself is quite simple:

- ensure you have set the dns server at your dns registrar to cloudflare [LINK](#)
- create cloudflare api token [LINK](#)

After this you can start an example docker compose project which will retrieve ssl certs for your domain using your token Use [this repository as example](#). This example will start a traefik reverse proxy which will acquire the certificate based on its configuration.

SSL / TLS

TLS, short for Transport Layer Security, and [SSL](#), short for Secure Socket Layers, are both cryptographic protocols that encrypt data and authenticate a connection when moving data on the Internet.

Well, **TLS is actually just a more recent version of SSL**

- SSL 1.0 – never publicly released due to security issues.
- SSL 2.0 – released in 1995. Deprecated in 2011. Has known security issues.
- SSL 3.0 – released in 1996. Deprecated in 2015. Has known security issues.

- TLS 1.0 – released in 1999 as an upgrade to SSL 3.0. Planned deprecation in 2020.
- TLS 1.1 – released in 2006. Planned deprecation in 2020.
- TLS 1.2 – released in 2008.
- TLS 1.3 – released in 2018.

Why do we call the certs SSL cert? The reason why most people still refer to them as SSL certificates is basically a branding issue. Most major certificate providers still refer to certificates as SSL certificates, which is why the naming convention persists.

In reality, all the “SSL Certificates” that you see advertised are really **SSL/TLS Certificates**

HTTPS

What is HTTPS?! This is also where HTTPS comes in (HTTPS stands for “HTTP over SSL/TLS”).

What will be encrypted during a https request?

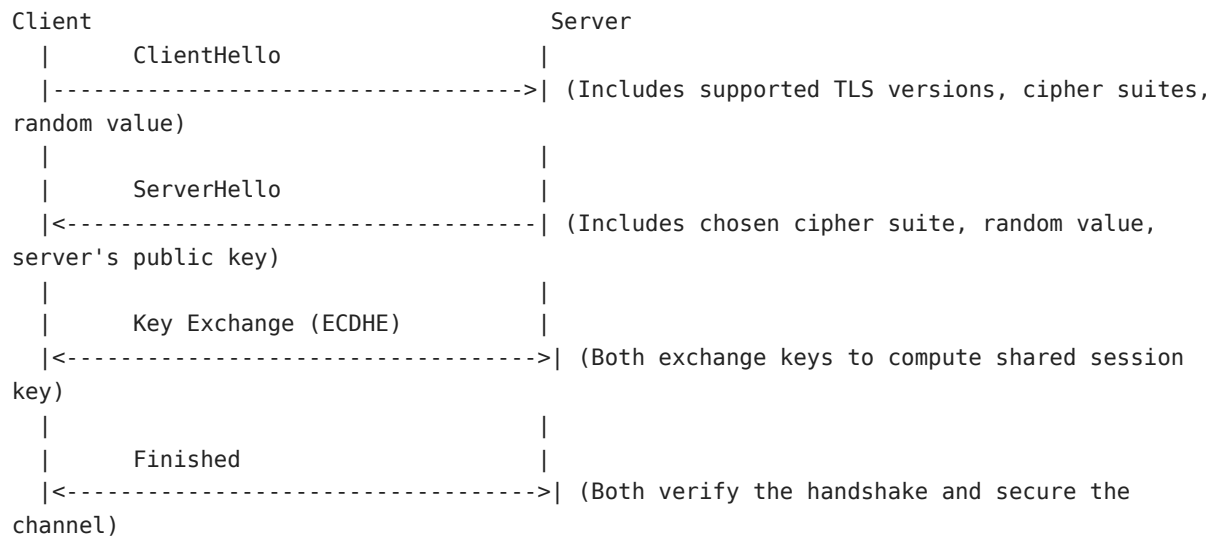
- request/response body
- headers
- query parameters

What will not be encrypted?

- hostname/domain
- port number
- path

HTTPS internal the TLS handshake

The TLS handshake is the process how the client (browser, curl etc..) and the server establish a secure communication over TLS protocol. **In a traditional TLS handshake process only the server proves its identity for the client with its certificate.**



You can debug this flow with one of this commands:

openssl:

```
openssl s_client -connect <server>:443 -msg -
debug
```

curl in verbose mode:

```
curl -vvv
https://<server>
```

or even with ssh

```
ssh -vvv
user@hostname
```

SSL termination

SSL termination is a process of terminating the SSL/TLS connection on the edge of your infrastructure. This way helping the management of certificates since you don't have to maintain them outside of your infrastructure's secure context. Helps to debug and trace the interservice communication. Helps to easy the load pressure on servers since they don't have to deal with encryption. SSL termination is usually done in the main ingress for your services.

Mutual TLS

It is an extension for TLS protocol which make possible that both parties authenticate to each other during the handshake process.

mTLS handshake process

Client	Server
ClientHello	
----->	(Includes supported TLS versions, cipher suites, random value)
ServerHello	
<-----	(Includes chosen cipher suite, random value, server's public key)
Certificate (Server's cert)	
<-----	(Server presents its certificate for authentication)
Key Exchange (ECDHE)	
<----->	(Both exchange keys to compute shared session key)
CertificateRequest	
<-----	(Server requests client's certificate for authentication)
Certificate (Client's cert)	
----->	(Client presents its certificate for authentication)
Finished	
<----->	(Both verify the handshake and secure the channel)

Lets try it out this mTLS flow!

CA

Create the CA key

```
openssl genpkey -algorithm RSA -out ca.key -aes256
```

Create the CA cert

```
openssl req -key ca.key -new -x509 -out  
ca.crt
```

Trust the new CA

```
sudo cp ca.crt /usr/local/share/ca-  
certificates/  
sudo update-ca-certificates
```

Server

Create the server key

```
openssl genpkey -algorithm RSA -out  
server.key
```

Create CSR for the server cert

```
openssl req -key server.key -new -out  
server.csr
```

Create cert with csr and key for the server

```
openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out server.crt -  
days 365
```

Client

Create the client key

```
openssl genpkey -algorithm RSA -out  
client.key
```

Create the CSR for the client

```
openssl req -key client.key -new -out  
client.csr
```

Create cert with CSR and key for the client

```
openssl x509 -req -in client.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out client.crt -  
days 365
```

Try it out

Start the test server with openssl

```
openssl s_server -key server.key -cert server.crt -CAfile ca.crt -Verify 1 -accept  
4433
```

Test it with curl

```
curl -vvv --cert client.crt --key client.key --cacert ca.crt  
https://localhost:4433
```

You can see the connection is properly created with the client certs

```

> curl -vvv --cert client.crt --key client.key --cacert ca.crt
https://localhost:4433
* Trying 127.0.0.1:4433...
* Connected to localhost (127.0.0.1) port 4433 (#0)
* ALPN, offering h2
* ALPN, offering http/1.1
* CAfile: ca.crt
* CPath: /etc/ssl/certs
* TLSv1.0 (OUT), TLS header, Certificate Status (22):
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
* TLSv1.2 (IN), TLS header, Certificate Status (22):
* TLSv1.3 (IN), TLS handshake, Server hello (2):
* TLSv1.2 (IN), TLS header, Finished (20):
* TLSv1.2 (IN), TLS header, Supplemental data (23):
* TLSv1.3 (IN), TLS handshake, Encrypted Extensions (8):
* TLSv1.2 (IN), TLS header, Supplemental data (23):
* TLSv1.3 (IN), TLS handshake, Request CERT (13):
* TLSv1.2 (IN), TLS header, Supplemental data (23):
* TLSv1.3 (IN), TLS handshake, Certificate (11):
* TLSv1.2 (IN), TLS header, Supplemental data (23):
* TLSv1.3 (IN), TLS handshake, CERT verify (15):
* TLSv1.2 (IN), TLS header, Supplemental data (23):
* TLSv1.3 (IN), TLS handshake, Finished (20):
* TLSv1.2 (OUT), TLS header, Finished (20):
* TLSv1.3 (OUT), TLS change cipher, Change cipher spec (1):
* TLSv1.2 (OUT), TLS header, Supplemental data (23):
* TLSv1.3 (OUT), TLS handshake, Certificate (11):
* TLSv1.2 (OUT), TLS header, Supplemental data (23):
* TLSv1.3 (OUT), TLS handshake, CERT verify (15):
* TLSv1.2 (OUT), TLS header, Supplemental data (23):
* TLSv1.3 (OUT), TLS handshake, Finished (20):
* SSL connection using TLSv1.3 / TLS_AES_256_GCM_SHA384
* ALPN, server did not agree to a protocol
* Server certificate:
* subject: C=AU; ST=Some-State; O=Internet Widgits Pty Ltd; CN=localhost
* start date: Nov 23 12:06:50 2024 GMT
* expire date: Nov 23 12:06:50 2025 GMT
* common name: localhost (matched)
* issuer: C=AU; ST=Some-State; O=Internet Widgits Pty Ltd; CN=localhost
* SSL certificate verify ok.
* TLSv1.2 (OUT), TLS header, Supplemental data (23):
> GET / HTTP/1.1
> Host: localhost:4433
> User-Agent: curl/7.81.0
> Accept: */*
>
* TLSv1.2 (IN), TLS header, Supplemental data (23):
* TLSv1.3 (IN), TLS handshake, Newsession Ticket (4):
* TLSv1.2 (IN), TLS header, Supplemental data (23):
* TLSv1.3 (IN), TLS handshake, Newsession Ticket (4):

```

However when you try it without client cert it fails:

```

curl -vvv https://localhost:4433
* Trying 127.0.0.1:4433...
* Connected to localhost (127.0.0.1) port 4433 (#0)
* ALPN, offering h2
* ALPN, offering http/1.1
* CAfile: /etc/ssl/certs/ca-certificates.crt
* CAPath: /etc/ssl/certs
* TLSv1.0 (OUT), TLS header, Certificate Status (22):
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
* TLSv1.2 (IN), TLS header, Certificate Status (22):
* TLSv1.3 (IN), TLS handshake, Server hello (2):
* TLSv1.2 (IN), TLS header, Finished (20):
* TLSv1.2 (IN), TLS header, Supplemental data (23):
* TLSv1.3 (IN), TLS handshake, Encrypted Extensions (8):
* TLSv1.2 (IN), TLS header, Supplemental data (23):
* TLSv1.3 (IN), TLS handshake, Request CERT (13):
* TLSv1.2 (IN), TLS header, Supplemental data (23):
* TLSv1.3 (IN), TLS handshake, Certificate (11):
* TLSv1.2 (IN), TLS header, Supplemental data (23):
* TLSv1.3 (IN), TLS handshake, CERT verify (15):
* TLSv1.2 (IN), TLS header, Supplemental data (23):
* TLSv1.3 (IN), TLS handshake, Finished (20):
* TLSv1.2 (OUT), TLS header, Finished (20):
* TLSv1.3 (OUT), TLS change cipher, Change cipher spec (1):
* TLSv1.2 (OUT), TLS header, Supplemental data (23):
* TLSv1.3 (OUT), TLS handshake, Certificate (11):
* TLSv1.2 (OUT), TLS header, Supplemental data (23):
* TLSv1.3 (OUT), TLS handshake, Finished (20):
* SSL connection using TLSv1.3 / TLS_AES_256_GCM_SHA384
* ALPN, server did not agree to a protocol
* Server certificate:
* subject: C=AU; ST=Some-State; O=Internet Widgits Pty Ltd; CN=localhost
* start date: Nov 23 12:06:50 2024 GMT
* expire date: Nov 23 12:06:50 2025 GMT
* common name: localhost (matched)
* issuer: C=AU; ST=Some-State; O=Internet Widgits Pty Ltd; CN=localhost
* SSL certificate verify ok.
* TLSv1.2 (OUT), TLS header, Supplemental data (23):
> GET / HTTP/1.1
> Host: localhost:4433
> User-Agent: curl/7.81.0
> Accept: */*
>
* TLSv1.2 (IN), TLS header, Supplemental data (23):
* TLSv1.3 (IN), TLS alert, unknown (628):
* OpenSSL SSL_read: error:0A00045C:SSL routines::tlsv13 alert certificate required, errno 0
* Closing connection 0
curl: (56) OpenSSL SSL_read: error:0A00045C:SSL routines::tlsv13 alert certificate
required, errno 0

```

Thanks for keeping with me!

It was a pleasure sharing this journey with you. I have tried to cover every certificate-related topic I have encountered during my career so far. If you feel that anything is missing, I would greatly appreciate your feedback.

Have a wonderful day! 🌞